

FEN Schema for C-SQD

Marcus Thomas

June 7, 2026

Contents

1 Overview	2
2 Domain Instantiation	2
2.1 Domain Configuration	2
2.2 The CWE Taxonomy	3
3 Audit Objects	3
3.1 AuditObject Lineage	4
4 Users	4
4.1 Reviewer Profile	5
5 Review Events	6
5.1 ReviewEvent Payload	7
5.2 ReviewEvent Membership	8
6 ER Solicitations	9
6.1 Solicitation Lifecycle	9
7 Synthesis Reviews	10
7.1 Synthesis Review Relations	10
8 Challenges	11
9 The Evaluation Tuple	11
9.1 Tuple Structure	11
9.2 Domain-Specific Operationalization	12
9.3 Computation Signature	12
10 Shared Value Types	13
11 Traits	13
12 Summary	14

1 Overview

This document specifies the internal data schema for C-SQD. The schema is adapted from the Facts, Episodes, and Narratives (FEN) schema developed for Phoros. The FEN schema organizes clinical data into three layers: atomic observations (*Facts*), problem-centric groupings (*Episodes*), and authored interpretations (*Narratives*). These layers map onto C-SQD’s epistemic auditing architecture with structural fidelity.

The mapping is as follows. Facts become **ReviewEvents**: atomic, timestamped, provenance-bearing evaluative acts. Episodes become **AuditObjects**: persistent accumulations of ReviewEvents organized around a single artifact. Narratives become **SynthesisReviews**: authored integrative interpretations that can coexist, contest each other, and supersede each other.

C-SQD extends FEN in two directions. First, it introduces **DomainInstantiation** as a first-class layer above AuditObjects, capturing the fact that C-SQD is a general epistemic audit architecture instantiated in specific domains. Second, it introduces a computed **evaluation tuple** as a derived view over accumulated ReviewEvent records, dynamically recomputable by users.

The schema is defined as Rust types. Storage is abstracted.

2 Domain Instantiation

C-SQD may be instantiated in multiple domains — academic publishing, clinical trial review, AI auditing, policy review, and others. Each instantiation defines its own CWE taxonomy, evaluation tuple configuration, and review phase parameters. **DomainInstantiation** is a first-class entity that owns this configuration.

```
struct DomainInstantiationId(Uuid);

struct DomainInstantiation {
    id: DomainInstantiationId,
    domain_type: DomainType,
    config: DomainConfig,
    cwe: CWETaxonomy,
    created_at: Timestamp,
    governed_by: Principal,
}

enum DomainType {
    AcademicPublishing,
    ClinicalTrialReview,
    AIAuditing,
    PolicyReview,
    Custom(String),
}
```

Multiple instantiations of the same **DomainType** are permitted. A conference partnership program is one instantiation of **AcademicPublishing**; a sponsored NIH review program is another. Every downstream entity — AuditObjects, ReviewEvents, CWE criteria — carries a `domain_instantiation_id` that anchors it to its instantiation.

2.1 Domain Configuration

DomainConfig captures phase parameters and evaluation tuple configuration for a given instantiation.

```
struct DomainConfig {
    phase_config: PhaseConfig,
    eval_tuple_config: EvalTupleConfig,
    audit_object_types: Vec<AuditObjectType>,
    reviewer_concurrency: ReviewerConcurrencyLimits,
}
```

```
struct PhaseConfig {
    public_review_duration: Option<Duration>,
    response_rounds_permitted: u32,
    synthesis_significance_threshold: f64,
    anonymity_rules: AnonymityConfig,
}

struct ReviewerConcurrencyLimits {
    max_active_element_reviews: u32,
    max_active_synthesis_reviews: u32,
}
```

Phases may partially overlap. Phase state at any time T is a derived query over `PhaseConfig` parameters and the audit object's submission timestamp; it is not stored as mutable state.

2.2 The CWE Taxonomy

The Common Weakness Enumeration (CWE) is an extensible taxonomy of evaluation criteria. It is a node tree, not a closed enum. Each node carries provenance indicating whether it belongs to the base taxonomy or a community extension.

```
struct CWENodeId(Uuid);

struct CWENode {
    id: CWENodeId,
    domain_instantiation_id: DomainInstantiationId,
    parent: Option<CWENodeId>,
    label: String,
    description: String,
    source: CWESource,
}

enum CWESource {
    BaseTaxonomy,
    CommunityExtension { community_id: CommunityId },
    VerifiedTag { vtag_id: VTagId },
}
```

A `CWECriterionId` is a compound key that carries its domain context implicitly.

```
struct CWECriterionId {
    domain: DomainInstantiationId,
    node_id: CWENodeId,
}
```

3 Audit Objects

An **AuditObject** is any artifact containing claims that can be subjected to structured evaluation. It is the persistent accumulation point for `ReviewEvents`, analogous to `ProblemEpisode` in FEN.

```
struct AuditObjectId(Uuid);

struct AuditObject {
    id: AuditObjectId,
    domain_instantiation_id: DomainInstantiationId,
    object_type: AuditObjectType,
    submitted_by: UserId,
    submitted_at: Timestamp,
```

```

    status: AuditObjectStatus,
    submission_tier: SubmissionTier,
    external_refs: Vec<ExternalRef>,
}

enum AuditObjectStatus {
    Active,
    Revised {
        superseded_by: AuditObjectId,
        superseded_at: Timestamp,
    },
    Withdrawn {
        withdrawn_at: Timestamp,
        reason: Option<String>,
    },
}

enum SubmissionTier {
    Tier0,
    Tier1,
    Tier2,
    Tier3Plus,
}

```

3.1 AuditObject Lineage

Audit objects are revised, superseded, and split. `AuditObjectRelation` records these relationships explicitly. No relationship is inferred; every link is an authored claim with provenance.

```

struct AuditObjectRelation {
    id: RelationId,
    source: AuditObjectId,
    target: AuditObjectId,
    relation_type: AuditObjectRelationType,
    asserted_by: Principal,
    asserted_at: Timestamp,
}

enum AuditObjectRelationType {
    Supersedes,
    Revises,
    SplitFrom,
    MergedInto,
    RelatedTo,
}

```

4 Users

Every platform participant is a `User`. Reviewer and submitter are roles that context supplies, not distinct types. A `ReviewEvent` records who reviewed; an `AuditObject` records who submitted. No role annotation on `User` is needed. A user who has completed the steps to review carries a `ReviewerProfile`; one who has not carries `None`.

```

struct UserId(Uuid);

struct User {

```

```

    id: UserId,
    display_name: String,
    email: String,
    created_at: Timestamp,
    status: UserStatus,
    reviewer_profile: Option<ReviewerProfile>,
}

enum UserStatus {
    Active,
    Suspended { reason: String, until: Option<Timestamp> },
    Deactivated,
}

```

`Principal` is the reference type used in provenance contexts. It identifies who asserted something without carrying the full `User` record.

```

enum Principal {
    User(UserId),
    Platform,
    AiAssisted { tool_id: String, supervised_by: Option<UserId> },
}

```

4.1 Reviewer Profile

`ReviewerProfile` is a separate top-level entity keyed by `UserId`. It is fetched independently when reviewer-specific context is needed, keeping `User` lightweight. The profile is extensible: domain instantiations may add fields via `domain_extensions`.

```

struct ReviewerProfile {
    user_id: UserId,
    status: ReviewerStatus,
    tags: Vec<ReviewerTag>,
    endorsements: Vec<Endorsement>,
    professional_relations: Vec<ProfessionalRelation>,
    concurrency_state: ReviewerConcurrencyState,
    domain_extensions: Vec<ReviewerDomainExtension>,
}

enum ReviewerStatus {
    GracePeriod,
    Active,
    Penalized { severity: PenaltySeverity, until: Option<Timestamp> },
    Suspended,
}

struct ReviewerTag {
    id: TagId,
    label: String,
    scope: TagScope,
    verified: bool,
}

enum TagScope {
    Global,
    Domain(DomainInstantiationId),
    VerifiedTag { vtag_id: VTagId, domain: DomainInstantiationId },
}

```

```

struct Endorsement {
    id: EndorsementId,
    endorsed_by: UserId,
    domain: DomainInstantiationId,
    fields: Vec<String>,
    endorsed_at: Timestamp,
    public: bool,
}

struct ProfessionalRelation {
    relation_type: ProfessionalRelationType,
    counterpart: UserId,
    established_at: Timestamp,
}

enum ProfessionalRelationType {
    Collaborator,    // established by joint review authorship
    Mentor,
    Mentee,
}

struct ReviewerConcurrencyState {
    active_element_reviews: u32,
    active_synthesis_reviews: u32,
}

struct ReviewerDomainExtension {
    domain_instantiation_id: DomainInstantiationId,
    data: serde_json::Value,    // domain-specific fields
}

```

5 Review Events

A **ReviewEvent** is the atomic unit of evaluative activity in C-SQD. It is timestamped, authored, and provenance-bearing. ReviewEvents are not updated; if a ReviewEvent is wrong or superseded, its status changes and a new ReviewEvent is created.

```

struct ReviewEventId(Uuid);

struct ReviewEvent {
    id: ReviewEventId,
    audit_object_id: AuditObjectId,
    domain_instantiation_id: DomainInstantiationId,
    occurred_at: Timestamp,
    payload: ReviewEventPayload,
    status: ReviewEventStatus,
    provenance: Provenance,
}

enum ReviewEventStatus {
    Active,
    Superseded {
        superseded_by: Principal,
        superseded_at: Timestamp,
        replaced_by: Option<ReviewEventId>,
    },
    Retracted {

```

```

    retracted_by: Principal,
    retracted_at: Timestamp,
    reason: Option<String>,
  },
}

```

5.1 ReviewEvent Payload

ReviewEventPayload is a sum type. Each variant carries the type-specific data for that kind of evaluative act.

```

enum ReviewEventPayload {
  ElementReview {
    cwe_criterion: CWECriterionId,
    submitted_by: UserId,
    solicitation: Option<ERSolicitationId>,
    finding: Finding,
    severity: Option<FindingSeverity>,
    content: String,
    featured: bool,
  },
  SynthesisReview {
    submitted_by: UserId,
    content: String,
    sections: Vec<SynthesisSection>,
    featured: bool,
  },
  SubmitterResponse {
    responding_to: Vec<ReviewEventId>,
    response_type: ResponseType,
    content: String,
    revision_ref: Option<AuditObjectId>,
  },
  BountyPosting {
    posted_by: Principal,
    cwe_criterion: Option<CWECriterionId>,
    target_review: Option<ReviewEventId>, // non-null for challenge bounties
    amount: Money,
    conditions: String,
    expires_at: Option<Timestamp>,
  },
  BountySubmission {
    bounty_id: ReviewEventId,
    submitted_review: ReviewEventId,
    submitted_by: UserId,
  },
  BountyAdjudication {
    bounty_id: ReviewEventId,
    submission_id: ReviewEventId,
    outcome: AdjudicationOutcome,
    adjudicated_by: Principal,
  },
}

enum Finding {
  NonEthicalProblem,
  EthicalProblem,
  NoProblems,
}

```

```

        Inconclusive,
    }

    enum FindingSeverity {
        Minor,
        Moderate,
        Major,
        Critical,
    }

    enum ResponseType {
        Accepts,
        Contests,
        PartiallyAccepts,
        RevisesAndResponds,
    }

    enum AdjudicationOutcome {
        Awarded,
        Rejected,
        PartiallyAwarded,
        Expired,
    }

```

5.2 ReviewEvent Membership

`ReviewEventMembership` links a `ReviewEvent` to an `AuditObject`. It is a standalone struct, not a list on `AuditObject`. This design is inherited directly from FEN's `EpisodeMembership` and preserves its four advantages: granular contestability, multi-object membership, audit history, and role annotation.

```

struct MembershipId(Uuid);

struct ReviewEventMembership {
    id: MembershipId,
    review_event_id: ReviewEventId,
    audit_object_id: AuditObjectId,
    role: ReviewEventRole,
    asserted_by: Principal,
    asserted_at: Timestamp,
    status: MembershipStatus,
}

enum ReviewEventRole {
    ElementReview,
    SynthesisReview,
    SubmitterResponse,
    BountyPosting,
    BountySubmission,
    BountyAdjudication,
}

enum MembershipStatus {
    Active,
    Retracted {
        retracted_by: Principal,
        retracted_at: Timestamp,
    },
}

```


6 ER Solicitations

An **ERSolicitation** is a platform-issued assignment matching a reviewer to an audit object, a CWE criterion, and a payment scheme. It is the mechanism by which funded review is initiated. An unsolicited **ElementReview** carries `solicitation: None`; a solicited one references its `ERSolicitationId`.

```
struct ERSolicitationId(Uuid);

struct ERSolicitation {
    id: ERSolicitationId,
    audit_object_id: AuditObjectId,
    cwe_criterion: CWECriterionId,
    issued_to: UserId,
    payment_scheme: PaymentScheme,
    issued_at: Timestamp,
    domain_instantiation_id: DomainInstantiationId,
}

struct PaymentScheme {
    amount: Money,
    currency: String,
    condition: PaymentCondition,
}

enum PaymentCondition {
    OnSubmission,
    OnAcceptance,
    Tiered { on_submission: Money, on_acceptance: Money },
}
```

6.1 Solicitation Lifecycle

Solicitation state is not stored as a mutable field on **ERSolicitation**. Instead, lifecycle events are recorded as an append-only log of **SolicitationEvent** records. Current state is the `event_type` of the most recent event for a given solicitation. This design preserves the full history of the solicitation lifecycle and provides authoritative timestamps for reviewer penalty computation.

```
struct SolicitationEventId(Uuid);

struct SolicitationEvent {
    id: SolicitationEventId,
    solicitation_id: ERSolicitationId,
    event_type: SolicitationEventType,
    occurred_at: Timestamp,
    principal: Principal,
    note: Option<String>,
}

enum SolicitationEventType {
    Issued,
    Accepted,
    Declined,
    Expired,
    Completed,
    PenaltyFlagged { severity: PenaltySeverity },
}

enum PenaltySeverity {
```

```
Moderate,      // submission between 3 and 5 weeks after acceptance
Severe,        // submission beyond 6 weeks or no submission
EthicsViolation,
}
```

7 Synthesis Reviews

A `SynthesisReview` payload carries optional internal structure via `SynthesisSection`. Not all `SynthesisReviews` require sections; the top-level `content` field may suffice. Sections are useful when the review distinguishes phases of evaluation or references specific `ElementReviews`.

```
struct SynthesisSectionId(Uuid);

struct SynthesisSection {
    id: SynthesisSectionId,
    review_event_id: ReviewEventId,
    section_type: SynthesisSectionType,
    content: String,
    referenced_reviews: Vec<ReviewEventId>,
}

enum SynthesisSectionType {
    Summary,
    MethodologicalAssessment,
    EthicalAssessment,
    EvidenceIntegration,
    Recommendations,
    OpenQuestions,
}
```

7.1 Synthesis Review Relations

Multiple `SynthesisReviews` per audit object is the normal case. Reviewers may agree, partially overlap, or contest each other. `SynthesisReviewRelation` records these relationships explicitly.

```
struct SynthesisReviewRelation {
    id: RelationId,
    source: ReviewEventId,
    target: ReviewEventId,
    relation_type: SynthesisRelationType,
    asserted_by: Principal,
    asserted_at: Timestamp,
}

enum SynthesisRelationType {
    Supersedes,
    Contests(ContestationInfo),
    RelatedTo,
}

struct ContestationInfo {
    scope: ContestationScope,
    rationale: Option<String>,
}

enum ContestationScope {
    Partial,
```

```
    Full,
}
```

8 Challenges

The challenge system maintains a featured subset of ElementReviews and SynthesisReviews representing the best current evaluation of an audit object. **Challenge** provides the schema-level abstraction. Election mechanics, vote weighting, incumbency rules, and the 30-day window are specified in a companion document.

```
struct ChallengeId(Uuid);

struct Challenge {
    id: ChallengeId,
    challenge_type: ChallengeType,
    target: ChallengeTarget,
    challenger_review: Option<ReviewEventId>, // null for petition challenges
    initiated_by: Principal,
    initiated_at: Timestamp,
    election_date: Timestamp,
    status: ChallengeStatus,
    domain_instantiation_id: DomainInstantiationId,
}

enum ChallengeType {
    Direct,
    Petition,
}

enum ChallengeTarget {
    ElementReview(ReviewEventId),
    SynthesisReview(ReviewEventId),
}

enum ChallengeStatus {
    Open,
    Resolved(ChallengeOutcome),
    Withdrawn,
}

struct ChallengeOutcome {
    winner: ReviewEventId,
    decided_at: Timestamp,
}
```

9 The Evaluation Tuple

The evaluation tuple is a computed view, not a stored property. It is a pure function over accumulated ReviewEventMembership records, filtered by reviewer community and reference time. Its purity and statelessness over immutable records makes it suitable for client-side recomputation via WASM. The WASM compilation target and interface bindings are out of scope for this document.

9.1 Tuple Structure

$$E(A \mid R, T_{\text{eval}}) \rightarrow (N, M, S, L, U)$$

The function takes an audit object A , a reviewer community filter R , and a reference time T_{eval} . It returns a five-dimensional tuple where:

- N counts non-ethical problems identified by reviewers in R up to T_{eval}
- M counts ethical problems identified by reviewers in R up to T_{eval}
- S quantifies the stakes of the audit object, operationalized per domain instantiation
- L quantifies scrutiny depth, weighting ReviewEvents by reviewer expertise and solicitation status
- U tracks uptake of the audit object and its audit record into subsequent artifacts or decisions, operationalized per domain instantiation

9.2 Domain-Specific Operationalization

S and U vary in meaning across domain instantiations. EvalTupleConfig on DomainConfig captures this variation.

```
struct EvalTupleConfig {
    stakes_operationalization: StakesDefinition,
    uptake_operationalization: UptakeDefinition,
    l_weight_params: ScrutinyWeightParams,
}

enum StakesDefinition {
    ScientificSignificance,
    PublicHealthConsequentiality,
    DeploymentRiskProfile,
    Custom(String),
}

enum UptakeDefinition {
    CitationImpact,
    DownstreamAdoption,
    DeploymentDecisions,
    Custom(String),
}

struct ScrutinyWeightParams {
    solicited_review_multiplier: f64,
    bounty_triggered_multiplier: f64,
    expertise_weight_fn: String, // identifier for the weighting function
}
```

9.3 Computation Signature

```
fn compute_eval_tuple(
    audit_object_id: AuditObjectId,
    community: ReviewerCommunityFilter,
    t_eval: Timestamp,
    config: &EvalTupleConfig,
) -> EvalTuple

struct EvalTuple {
    n: f64,
    m: f64,
    s: f64,
```

```

    l: f64,
    u: f64,
    computed_at: Timestamp,
    community_filter: ReviewerCommunityFilter,
}

struct ReviewerCommunityFilter {
    tags: Vec<TagId>,
    domain_instantiation_id: Option<DomainInstantiationId>,
    min_endorsements: Option<u32>,
}

```

10 Shared Value Types

```

struct Provenance {
    source_system: Option<String>,
    source_document: Option<DocumentId>,
    imported_at: Timestamp,
    principal: Principal,
}

struct ExternalRef {
    system: ExternalSystem,
    resource_type: Option<String>,
    resource_id: String,
    uri: Option<String>,
}

enum ExternalSystem {
    DOI,
    ArXiv,
    ClinicalTrialsGov,
    ORCID,
    Other(String),
}

struct Money {
    amount: f64,
    currency: String,
}

```

11 Traits

The schema uses traits where the system needs polymorphic dispatch over shared behavior. Two traits are warranted.

Temporal The system must place heterogeneous objects on a single timeline. `ReviewEvent`, `SolicitationEvent`, `Challenge`, and `ReviewEventMembership` all implement this trait, enabling timeline views that interleave entity types without matching on every concrete type.

```

trait Temporal {
    fn temporal_anchor(&self) -> Timestamp;
}

```

Authored The system must answer provenance queries across entity types — audit logs, author-filtered views, and conflict disclosure all require knowing who asserted what and when across entity boundaries.

```
trait Authored {
  fn principal(&self) -> &Principal;
  fn authored_at(&self) -> Timestamp;
}
```

Structural overlap without behavioral need does not justify a trait. Where two structs share field names but are never collected into the same generic context, the fields are repeated rather than abstracted.

12 Summary

The C-SQD schema defines the following entity types across five layers:

Layer	Entities	Ontological status
Domain	DomainInstantiation, DomainConfig, CWENode	Configuration
Users	User, ReviewerProfile	Identity and reviewer state
Audit Objects	AuditObject, AuditObjectRelation	Structured artifact records
Review Events	ReviewEvent, ReviewEventMembership, ERSolicitation, SolicitationEvent, Challenge	Atomic evaluative acts
Interpretations	SynthesisSection, SynthesisReviewRelation	Authored integrations

Trait	Purpose
Temporal	Unified timeline placement across entity types
Authored	Cross-entity provenance queries

The evaluation tuple $E(A \mid R, T_{\text{eval}}) \rightarrow (N, M, S, L, U)$ is a derived view, not stored state. It is a pure function over immutable **ReviewEventMembership** records, recomputable client-side for any reviewer community and reference time.

The central structural decision, inherited from FEN, is that ReviewEvent-to-AuditObject assignment is a standalone provenance-bearing record rather than a list on the AuditObject. The central extension beyond FEN is that **DomainInstantiation** is a first-class entity that owns the CWE taxonomy and evaluation tuple configuration, making C-SQD’s domain-generalty explicit in the type system.